Practical 5D

Develop a Web Application to Perform CRUD Operations on an Employee Table Using EF Core

Aim

To create an ASP.NET Core MVC application that performs Create, Read, Update, and Delete (CRUD) operations on employee records using Entity Framework Core and SQL Server.

Procedure

1. Create a New MVC Project

1. Open Visual Studio 2026.
2. Select Create a new project.
3. Choose ASP.NET Core Web App (Model-View-Controller), then click Next.
4. Set Project name to EmployeeCRUD and choose a suitable location.
5. Select .NET 10.0 (or the latest framework available in your Visual Studio installation).
6. Click Create.

2. Install Entity Framework Core Packages

Open Tools → NuGet Package Manager → Manage NuGet Packages for Solution. Install compatible versions of the following packages. Use the same version for all EF Core packages; the reference practical specifies version 10.0.8 or a compatible version available in your environment.

- Microsoft.EntityFrameworkCore.SqlServer
- Microsoft.EntityFrameworkCore
- Microsoft.EntityFrameworkCore.Tools
- Microsoft.EntityFrameworkCore.Design
- Microsoft.VisualStudio.Web.CodeGeneration.Design

3. Create the Employee Model

In Solution Explorer, right-click Models → Add → Class. Name the file Employee.cs and enter:

```
namespace EmployeeCRUD.Models
{
    public class Employee
    {
        public int Id { get; set; }

        public required string Name { get; set; }

        public required string Department { get; set; }

        public double Salary { get; set; }
    }
}
```

4. Create the Data Folder

Right-click the project → Add → New Folder. Name the folder Data.

5. Create the DbContext Class

Right-click the Data folder → Add → Class. Name the file AppDbContext.cs and enter:

```
using EmployeeCRUD.Models;
using Microsoft.EntityFrameworkCore;

namespace EmployeeCRUD.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(
            DbContextOptions<AppDbContext> options)
            : base(options)
        {
        }

        public DbSet<Employee> Employees { get; set; }
    }
}
```

6. Add the Database Connection String

Open appsettings.json. Add a ConnectionStrings section. The LocalDB connection below follows the reference practical; ensure SQL Server LocalDB is installed. If you use another SQL Server instance, change the Server value accordingly.

```
{
  "ConnectionStrings": {
    "MyConnection":
"Server=(localdb)\\MSSQLLocalDB;Database=EmployeeDB;Trusted_Connection=True;TrustServerCert
ificate=True"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

7. Configure DbContext in Program.cs

Add the required using statements at the top of Program.cs. Register AppDbContext before builder.Build(), as shown:

```
using EmployeeCRUD.Data;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();
```

```
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("MyConnection")));

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

8. Create the Employee Controller Using Scaffolding

1. Build the project first.
2. In Solution Explorer, right-click Controllers → Add → New Scaffolded Item.
3. Select MVC Controller with views, using Entity Framework.
4. Click Add.
5. Select Model class: Employee (EmployeeCRUD.Models).
6. Select Data context class: AppDbContext (EmployeeCRUD.Data).
7. Keep the generated controller name as EmployeesController, then click Add.

Visual Studio generates the controller and views for the standard CRUD operations: Index, Details, Create, Edit, and Delete.

9. Create the Database

Open Tools → NuGet Package Manager → Package Manager Console. Run the following commands one at a time:

```
Add-Migration InitialCreate
Update-Database
```

If the migration reports a package advisory or warning, first check whether it is only a warning or an actual build/migration error. Update packages to compatible versions where possible, then rebuild the project and run the migration commands again. Avoid adding an advisory-suppression entry unless your instructor or project administrator specifically requires it.

10. Run and Test the Application

1. Build the solution: Build → Build Solution.
2. Run the project using HTTPS or IIS Express.
3. In the browser address bar, append /Employees to the application URL. For example: https://localhost:7000/Employees (use the port shown on your own machine).
4. Test the following operations:

- Create: add an employee with name, department, and salary.
- Read: verify that the employee appears in the list and open Details.
- Update: select Edit, change employee information, and save.
- Delete: select Delete and confirm removal.

Expected Result

The EmployeeCRUD application runs successfully and displays an employee list. Users can create, view, edit, and delete employee records stored in the EmployeeDB database through Entity Framework Core.

Important Notes

- Make sure the project builds successfully before running migrations.
- Use a connection string that matches the SQL Server instance installed on your computer.
- If your installed .NET version differs from .NET 10, choose a framework supported by your Visual Studio installation and use compatible EF Core package versions.
- The generated EmployeesController and Razor views provide the CRUD interface.